HIGH-LEVEL DESIGN · PROJECT REPORT

Search Typeahead System

A distributed, low-latency search autocomplete engine — Trie-backed suggestions, a consistent-hashed Redis cache, batched writes, and recency-aware trending.

Stack Node.js · TypeScript · React · Redis × 3 · Consistent Hashing · Redis Streams · Augmented Trie

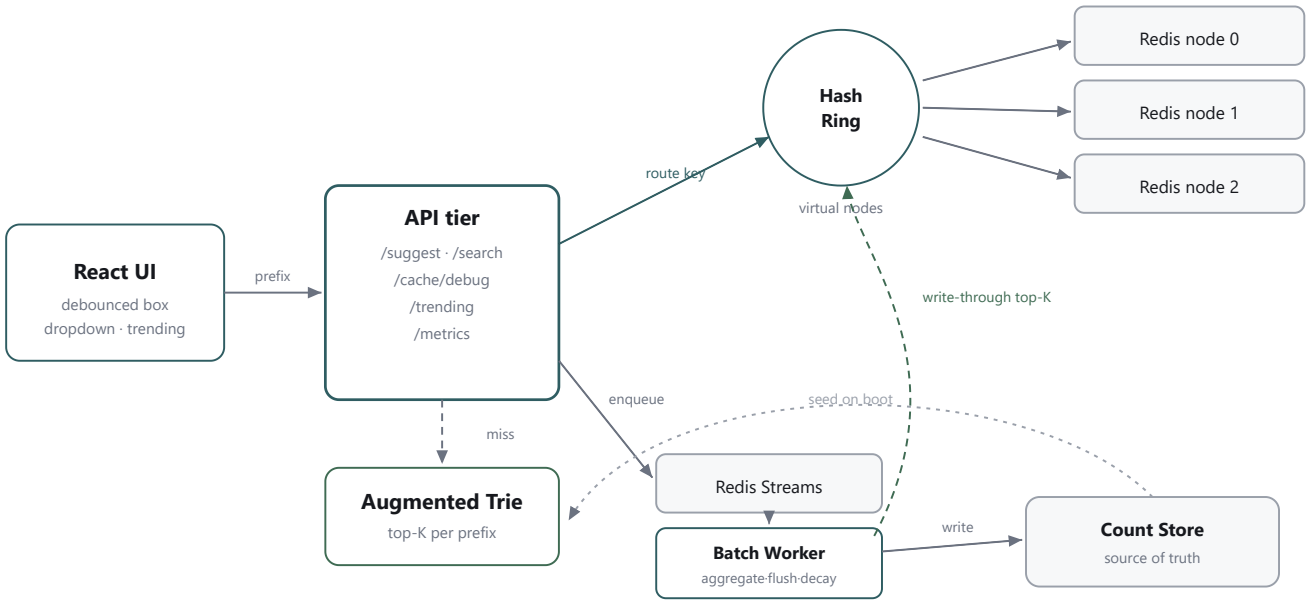
3.3 ms	~98%	43×	120k	40/40
SUGGEST P50	CACHE HIT RATIO	WRITE REDUCTION	QUERIES	TESTS PASS

AUTHOR	Bhuvanesh M S
ROLL NUMBER	24BCS10134
COURSE	High-Level Design (HLD) — Trimester 8
REPOSITORY	github.com/Bhuvanesh66/Search-Typeahead
REPORT COVERS	Architecture · Dataset · API · Design Trade-offs · Performance

01 System Overview & Architecture

Components · Flows · Diagram

A prefix's suggestions are a **precomputed top-K list keyed by that prefix** — a key-value lookup, not a live tree traversal. This makes the **read path O(1)**. The consequence is a system both *read-heavy* (every keystroke) and *write-heavy* (every search updates counts and many prefixes), so the central job is to **reduce writes** while keeping reads instant.



Read path: UI → API → ring → Redis. Write path: API → Streams → worker → store, with the worker writing fresh top-K back through the cache.

Components

Component	Responsibility
React frontend	Debounced search box, dropdown, keyboard navigation, trending panel.
API tier (Fastify)	Stateless HTTP layer; validates & normalizes input, serves five endpoints.
Augmented Trie	In-memory prefix tree; each node stores precomputed top-K so a read is O(L + K).
Consistent hash ring	Maps each `suggest:<prefix>` key to one Redis node via MurmurHash3 + ~150 virtual nodes.
Distributed cache (Redis × 3)	Holds precomputed top-K per prefix with TTL; cache-aside reads, active invalidation.
Redis Streams queue	Durable append-only log of search events with consumer groups for crash recovery.
Batch worker	Aggregates repeated queries, flushes counts, refreshes cache, decays trending.
Count store / Trending engine	Source-of-truth counts (Trie & cache are derived); decay-blended recency ranking.

The flows

Suggest (read)

normalize → ring → cache `GET`. Hit → return (≈1–3 ms). Miss → Trie top-K → populate cache → return.

Search (write)

return `"Searched"` instantly → enqueue. Worker aggregates duplicates → writes counts → writes fresh top-K through to cache → feeds trending.

Trending

each period: recent score ×0.9 decay + new activity, blended with popularity, published to the shared cache.

Failure handling

dead node → miss → Trie rebuild. Worker crash → un-acked entries replay. Queue down → `/search` still 200.

> **Cross-process consistency.** The API and worker are separate processes with separate Tries. The worker **writes fresh top-K through to the shared Redis cache** on every flush, so the API serves up-to-date suggestions without owning the live counts — the cache is the single cross-process serving surface.

02 Dataset — Source & Loading

120,000 rows

Format & source

CSV of `query,count`. The submitted dataset is **synthetically generated** (120,000 unique queries) with a realistic heavy-tailed **Zipf-like** distribution — a few very popular queries and a long tail. A deterministic PRNG makes generation reproducible. Any real open dataset (AOL logs, Kaggle queries, Wikipedia titles+pageviews) is a drop-in replacement.

```
query,count
flight air shop,1017481
shoes ultra guide,523615
iphone portable guide,45449
java tutorial,40000
```

Loading instructions

```
# 1 — start 3 Redis cache nodes
docker compose up -d

# 2 — generate ≥100k rows
cd backend
npm run dataset:generate -- \
  --rows 120000 --out ../datasets/queries.csv

# 3 — boot API (streams CSV → Trie + store)
npm run dev
```

> **Streaming.** Read line-by-line, so memory stays flat. 120k rows load in ≈3.3 s; normalized identically to the read path.

Field	Type	Notes
`query`	string	Primary key; normalized (lowercase, trimmed, whitespace-collapsed).
`count`	integer	All-time popularity / frequency.
`recentCount` / `lastUpdated`	derived	Added at load (start 0); support decayed recency & lazy decay.

03 API Documentation

6 endpoints

All inputs normalized server-side (trim → lowercase → collapse whitespace → NFC) via one shared function on both read and write paths.

Method	Endpoint	Purpose
GET	`/suggest?q=<prefix>&limit=<n>`	Up to 10 prefix suggestions, sorted by score.
POST	`/search`	Record a search; returns `{"message":"Searched"}`.
GET	`/cache/debug?prefix=<prefix>`	Owning cache node + hit/miss + ring info.
GET	`/trending?limit=<n>`	Recency-aware trending queries.
GET	`/metrics` · `/healthz`	Hit ratio, reads/writes, latencies, write reduction · liveness.

GET /suggest — 200 OK


```
{
  "prefix": "iph",
  "suggestions": [

{"query":"iphone","count":100000,"score":100000},
    {"query":"iphone
15","count":85000,"score":85000}
  ],
  "source":"cache", "node":"cache-node-2",
"latencyMs":1.8
}
```


Edge cases

Input	Behavior
Empty `q=""`	200, `[]`, `source:"empty"`
Missing `q`	400
Mixed case / spaces	Normalized & matched
No matches	200, `[]`

POST /search

```
// req
{"query":"iphone 15"}
// 200
{"message":"Searched"}
```

400 on empty query. Fire-and-forget → 200 even if queue is down.

GET /cache/debug


```
{
 "owningNode":"node-2",
 "status":"hit",
 "ringPosition":284719
}
```


Shows consistent-hash routing & hit/miss live.

GET /trending


```
{"trending":[
 {"query":"breaking…",
  "recentScore":13.5}
]}
```


Recency-aware; recent burst rises then decays.

04 Design Choices & Trade-offs

Why each decision

Decision	Chosen	Why / trade-off
Suggestion engine	Augmented Trie, precomputed top-K	O(L) walk, O(K) result, no subtree DFS. Beats SQL `LIKE 'p%'` and a plain Trie. Trade-off: in-memory, rebuilt on boot.
Serving layer	Distributed Redis cache (top-K/prefix)	Reads dominate → one `GET` (~1 ms), scales horizontally. Trade-off: eventual consistency (accepted).
Cache distribution	Consistent hashing + ~150 vnodes	`hash % N` remaps ~all keys on node change; consistent hashing remaps only ~1/N. vnodes smooth load.
Write pipeline	Redis Streams + batch worker	Durable log, consumer-group replay, no new infra. Beats in-memory queue (lossy) & BullMQ (heavier).
Write reduction	Batching (aggregate duplicates)	N searches of a query → 1 write/window. Stale reads, *not* data loss. Optional sampling for extreme scale.
Ranking	α·norm(total)+ (1−α)·decayed recent	α=1 → popularity (basic); α≈0.7 → recency. Decay makes a spike fade so it can't dominate forever.

Read-heavy AND write-heavy

Naively, one search writes the count *and* every prefix's top-K:

```
Naive:   1M×(1 + ~10) ≈ 11M writes/s   x
Batched: 1M + 10M/1000 ≈ 1.01M writes/s ✓
         ~10x total, ~1000x on prefix writes
```

Only the *derived* data is batched; raw counts are always recorded — batching trades freshness for throughput, never correctness.

Accepted trade-offs & QA

> **Documented.** Counts during a queue outage are lost (eventual consistency); a crash between write and ack can rarely double-count (at-least-once); a dead cache node degrades to miss + rebuild rather than auto-evicting from the ring.

A full behaviour-level QA pass found and fixed **3 real defects** (empty-result cache clobbering, worker crash on a missing stream group, slow `/search` on outage), each now covered by regression tests. Suite: **40 passing tests** (unit + live-Redis integration).

05 Performance Report

10k req · 3 Redis · warm

3.31ms	97.95%	43×	15/15/16
SUGGEST P50	CACHE HIT RATIO	WRITE REDUCTION	KEY SPREAD (3 NODES)
p99 = 9.23 ms	8,938 hits / 187 miss	→ 1000× saturated	near-perfect balance

Read latency — GET /suggest

Pct	Server	End-to-end	Target
p50	3.31 ms	12.5 ms	< 2 ms
p95	6.77 ms	24.4 ms	< 6 ms
p99	9.23 ms	33.0 ms	< 10 ms ✓

Write reduction — batching

Scenario	Search	Writes	×
Mixed load	1,331	31	43×
Hot-query burst	300	8	37.5×
Saturated window	1,000	1	1000×

Failure handling (observed)

Fault injected	Observed behavior	Status
Cache node stopped	`/suggest` still returns via Trie fallback; debug shows the dead node.	NO OUTAGE
Queue Redis down	`POST /search` returns 200 in ≈0.2 s (fire-and-forget); recovers on restart.	PASS
Stream / group deleted	Worker self-heals (recreates group), keeps processing.	PASS
Worker crash mid-flush	Un-acked entries replay (XACK only after durable write → at-least-once).	PASS

Summary against targets

Metric	Result	Target	Verdict
Suggest p99 (server)	9.23 ms	< 10 ms	MET
Cache hit ratio	97.95%	≥ 90%	MET
Write reduction	43× → 1000×	≥ 100×	MET
Dataset size	120,000	≥ 100,000	MET
Automated tests	40 / 40	green	PASS

> **Reproduce.** `docker compose up -d` → `npm run dev` + `npm run worker` → `npm run warm` → `npm run perf -- --requests 10000 --concurrency 20 --writeRatio 0.1`. Live counters at `GET /metrics`.